REVERSE ENGINEERING REPORT · 逆向解剖报告

AI 是如何「控制微信」的？_

一份写给所有人看的逆向工程报告：
拆解某 AI 营销平台「个人微信销售员工」功能的完整实现原理

人人能懂 | 技术+非技术双版本 | 证据可复核 | 已脱敏

4 层	18 个	10 条	3 条
端云架构	控制接口	代码铁证	路线对比

分析对象：某 AI 营销决策平台（SaaS）的微信自动化功能 | 分析方式：前端代码逆向 + 实测抓包
报告日期：2026 年 10 月 9 日 | 内容已脱敏，不涉及任何真实品牌与域名

simon · 一个不懂代码的 AI 极客
微信交流群 simonsxxs | GitHub github.com/simonsxxs | Gitee gitee.com/simonsxx

AI REVERSE ENGINEERING

一分钟看懂 FOR EVERYONE

一句话结论

它没有任何"黑魔法"——**所谓 AI 控制微信，是在你的 Windows 电脑上装了一个"代驾司机"（一个小程序助手），这个司机用自动化工具，替你盯微信窗口、读消息、打字、点按钮。**AI 只负责"想说什么"，真正"动手操作微信"的是跑在你电脑上的自动化脚本。

一个类比

把微信想象成一辆车。**云端的 AI** 是坐在指挥中心里决策的"大脑"，它决定什么时候回复、回复什么；但指挥中心碰不到你的车。**你电脑上的"助手"（本地代理）**才是坐进驾驶座的代驾——它通过一把"万能钥匙"（Windows 的无障碍接口，本来是给读屏软件用的）摸到微信的方向盘：看屏幕、找按钮、模拟打字。而这把钥匙对应的工具，是一个叫 `wxauto` 的开源 Python 库。

为什么我们能知道得这么确切?

该产品的前端代码里，藏着一套"错误翻译表"——把技术报错翻译成用户能看懂的提示。但翻译规则本身暴露了一切：`把 "wxauto" 改名叫 "微信助手"`、`把 "Weixin.exe" 改叫 "微信"`、还有"请保持微信主界面可见，同步期间勿操作鼠标键盘"、"开启 Windows 讲述人"、"将微信降级至 4.1.8" 这样的话术。这些句子只有一种技术方案会说：**界面自动化（RPA）**。

三个常见误解，一次澄清：它**不是**破解了微信的通信协议（没有"协议号"特征）；它**不是**腾讯官方支持的接口（个人微信没有官方 API）；它也**不是**独立运行在云端的机器人——它必须装在**一台开着微信的 Windows 电脑**上，离开你的电脑它什么都做不了。

它能替你做什么 CAPABILITIES

产品名叫「个人微信销售专员 · AI 员工」，页面宣称 7×24 小时自动获客、沟通、成交。四大能力中心：

能力中心	做的事	对应的关键接口
AI 接待中心	自动接待客户咨询，按知识库话术智能回复	`wechat/send` `wechat/knowledge-config`
客户增长中心	自动添加客户、自动通过好友申请（异步任务+轮询结果）	`wechat/add-friend` `wechat/accept-friend`
私域运营中心	朋友圈发布与互动（素材来自平台的 AI 图片管线）	`weixin/moments`
智能跟进中心	主动触达跟进、打标签/写备注 → 识别"意向客户"	`weixin/set-label` `weixin/set-remark`

怎么实现的 ARCHITECTURE

1

云端编排（SaaS 服务端）

18 个控制接口：启动/停止/状态/发消息/加好友/朋友圈/打标签…… 它是"指挥中心"：下发指令、汇总状态、存储 AI 话术（挂接知识库 RAG）。

▲ WebSocket 长连接（指令下发 / 状态回传） | 安装包也由云端分发（带进度条） ▼

2

本地代理（用户 Windows 电脑上的常驻服务）

页面点「下载微信插件」后安装的 Python 服务（uvicorn/FastAPI），监听本机 `127.0.0.1:8866` 端口。同一套本地服务还承载视频生成网关与本地知识库——与远程知识库构成"本地/远程"双模式。

▼ 调用

3

自动化引擎 wxauto / wxauto4

开源 Python 库，通过 Windows 无障碍接口（COM UIAutomation）驱动微信窗口。产品把它的报错统一改名叫"微信助手"——这正是暴露实现的关键证据。

▼ 模拟点击 / 读取界面

4

微信 PC 客户端（Weixin.exe，4.x）

被操作的对象。它自己不知道被自动化了——在它眼里，一切操作都像"人"在点。

证据：十条铁证 EVIDENCE

全部来自该产品前端代码（主程序包 8.7MB）。最有力的是一套"错误翻译表"——产品把技术报错翻译成用户话术，但翻译规则本身暴露了技术底牌：

证据	代码原文（节选）	说明了什么
自动化库改名	`replace(/wxauto4?/gi,"微信助手")`	用的是开源库 wxauto，界面改名掩饰
目标程序	`replace(/Weixin\.exe/gi,"微信")`	操作对象是微信 PC 客户端进程
本地服务	`"（8866），请确认本地服务已启动"` 、 `uvicorn`	用户电脑上跑着 Python 常驻服务
COM 自动化	`CoInitialize` 、 `Find Control Timeout` 、 `SetFocus`	Windows 无障碍接口驱动界面
Python 环境	`No module named tkinter`	本地有 Python 运行环境
运行约束	`"请保持微信主界面可见，同步期间勿操作鼠标键盘"`	抢焦点式自动化，期间人不能用电脑
无障碍依赖	`"开启 Windows「讲述人」"` 、 `"微信降级至 4.1.8"`	依赖系统无障碍树；微信版本被锁死
数据回流	`wechat_chat2` 输出文件、退出码、解析失败	联系人导出经本地中转文件
消息机制	`"自动回复…轮询…连接失败"`	靠轮询收发，非实时推送
字段识别	`/微信号|微信|账号|手机|phone|wxid/i`	自动识别联系人字段

给非技术读者：上面这些"代码原文"相当于机器人鞋底沾的泥——产品方把脸洗干净了（界面全是"微信助手"），但鞋底的泥（报错原文）说明它从哪条路来。

运行全流程 WORKFLOW

1. **装助手：**点「下载微信插件」→ 云端下发本地代理安装包（进度条）→ 本机装好 Python 常驻服务，并与云端建立长连接。
2. **开微信：**启动微信 PC 客户端（须是兼容的 4.1.8 版本）并登录，保持主窗口可见。
3. **点启动：**页面点「启动助手」→ 云端记录状态，本地代理开始通过 wxauto 附着微信窗口。
4. **自动接待：**轮询新消息 → 上云 → AI 按知识库生成回复 → 本地模拟键入发送。全程微信窗口不能被遮挡，鼠标键盘被占用。
5. **同步人脉：**通讯录遍历导出（经本地中转文件）→ 回传云端，进入客户管理。
6. **增值动作：**自动加好友/通过好友、朋友圈发布、打标签写备注 → 沉淀"意向客户"，形成获客–沟通–成交闭环。

三条技术路线对比 COMPARISON

最稳

企业微信官方 API

腾讯官方支持，体验最稳。但功能受官方接口面约束，且只能管企业微信。

该系统：开发中（企业微信路由已铺好但无权限）。

本报告主角

wxauto 桌面 RPA

该系统的选择。门槛低、上手快，代价是：微信窗口必须前台可见、键盘鼠标被占用、微信版本锁死 4.1.8、仅限 Windows、有封号风险。

高危

协议号模拟

伪装成微信设备直连服务器，全自动化程度最高。但属灰产高危地带，法律与封号风险最大——该系统明确没有走这条路。

这套架构里，最值得复用的不是"控制微信"

而是它的**端云协同骨架**：云端负责决策与数据，本机代理负责执行（同一代理还承载视频生成、本地知识库、云服务网关），安装包云端分发带进度，双知识库按场景切换（本地低延迟 / 远程大算力）。这套骨架与微信无关，任何需要"本地执行能力"的产品都可以照搬。

结语

所谓"AI 控制微信"，拆开看并不神秘：**云端大脑负责想，本机代驾负责做**。看懂它的价值不在于模仿，而在于三点：识别这类产品的真实能力边界、评估自己使用此类工具的安全代价，以及借鉴那套可以脱离微信单独存在的端云协同架构。技术永远中立，怎么用、用在哪，才是真正的问题。

作者：simon

一个不懂代码的 AI 极客 · 用 AI 把别人产品的底裤看明白

微信交流群	simonsxxs
GitHub	github.com/simonsxxs
Gitee	gitee.com/simonsxx

免责声明：本报告基于公开可获取的前端代码做技术与学习目的分析，内容已脱敏（不涉及任何真实品牌、域名与账号信息），不构成对任何产品的使用建议。界面自动化操作个人微信存在违反平台协议与封号的风险，请读者自行判断与承担。